1. TASK

Assignment Requirements

- Develop a Flutter application using either **GetX** for state management.
- Implement **Login/Register functionality**.
- Implement a **Blog Management feature**.
- Create a clean and well-designed UI based on the provided reference design.

2. Project Setup & Dependencies

2.1 Dependencies

The pubspec.yaml file declares all third-party packages the app depends on:

Package	Purpose / Why It Was Used
get: ^4.7.3	GetX library - provides state management (Rx variables), routing (GetMaterialApp, named routes), dependency injection (Get.put / Get.find), and snackbar/dialog utilities all in one package.
http: ^1.6.0	Standard Dart HTTP client for making REST API calls (GET, POST, PUT, DELETE) to the MockAPI backend.
intl: ^0.20.2	Used in BlogCard and BlogDetailScreen to format DateTime objects into readable date strings like "May 14, 2025" using DateFormat.

3. Folder Structure

The project follows a feature-driven folder structure inside the lib/ directory. Each folder has a single responsibility:

```
lib/
├── controllers/
│   ├── auth_controller.dart
│   └── blog_controller.dart
├── models/
│   ├── blog_model.dart
│   └── user_model.dart
├── screens/
│   ├── home_screen.dart
│   ├── blogs_screen.dart
│   ├── add_blog_screen.dart
│   ├── edit_blog_screen.dart
│   ├── blog_detail_screen.dart
│   ├── login_screen.dart
│   └── register_screen.dart
├── services/
│   └── api_service.dart
├── utils/
│   ├── api_endpoints.dart
│   ├── app_constants.dart
│   └── app_routes.dart
├── widgets/
```

```
│   ├── app_bottom_nav_bar.dart
│   ├── app_text_field.dart
│   ├── auth_header.dart
│   ├── blog_card.dart
│   └── gradient_button.dart
└── main.dart
```

4. App Entry Point - main.dart

main.dart is the first file Flutter runs. It does three things before launching the UI:

4.1 Dependency Injection

Before runApp() is called, three objects are registered into GetX's dependency container using Get.put():

```
Get.put(ApiService());      // HTTP helper - registered first
Get.put(AuthController());  // Needs ApiService, registered second
Get.put(BlogController());  // Needs both above, registered third
```

Using Get.put() rather than creating objects inside widgets ensures that the same single instance is shared across the entire app. Any screen can later retrieve any of these with Get.find<T>().

4.2 Routing

All routes are declared centrally using getPages. The app starts at AppRoutes.login (/login). Every route string is defined as a constant in AppRoutes, so changing a path requires editing only one file.

5. Data Models

5.1 BlogModel

Located at lib/models/blog_model.dart. Represents a single blog post in the app.

Field	Description
id (String)	Unique identifier assigned by MockAPI. Used when sending PUT/DELETE requests.
title (String)	The blog post heading. Read from the "blog_name" key in the API response.
content (String)	The full body text of the blog.
author (String)	Name of the person who wrote it. Mapped from "Author_name" in the API.
createdAt (DateTime)	Timestamp of when the blog was created. Parsed using DateTime.tryParse().

fromJson() - deserialisation

The factory constructor BlogModel.fromJson() converts the raw Map from the API response into a typed Dart object. Because MockAPI does not guarantee consistent key names, the constructor uses a fallback chain:
This defensive approach prevents the app from crashing if the API returns data under a slightly different

key name. DateTime.tryParse() is used instead of DateTime.parse() so that malformed date strings return null, which then falls back to DateTime.now().

toJson() - serialisation

Used when sending POST and PUT requests. It maps the Dart object back to the exact keys that MockAPI expects:

copyWith()

A utility method that creates a new BlogModel from an existing one, replacing only the fields you specify. This is used inside BlogController.updateBlog() so that the original blog data is preserved for unchanged fields.

5.2 UserModel

Located at lib/models/user_model.dart. Represents a registered user.

Field	Description
id (String)	Assigned by MockAPI on registration.
name (String)	Full name of the user. Shown as a greeting on the home screen.
email (String)	Used as the login identifier. Compared case-insensitively during login.
password (String)	Stored as plain text in MockAPI (fine for demo purposes). Compared during login.

6. API Service

Located at lib/services/api_service.dart. This is the only file in the project that directly communicates with the MockAPI backend. All HTTP logic is centralised here, so no screen or controller ever imports the http package directly.

API Base URL

https://6a04af9caa826ca75c090ef1.mockapi.io

Endpoints: /users (user accounts)
/blogs (blog posts)

6.1 login() - [GET]

Since MockAPI does not have a native authentication endpoint, login is implemented by fetching all users with a GET request and searching the list for a matching email and password:

1. GET /users → returns the full list of registered users
2. Iterate the list to find one where:
   user.email.toLowerCase() == email.toLowerCase()
   AND user.password == password
3. If found → return that UserModel
4. If not found → return null (controller shows "Invalid credentials")

The email comparison is made case-insensitive using .toLowerCase() to prevent login failures caused by capitalisation differences.

6.2 register()- [POST]

Registration runs in two steps to prevent duplicate accounts:

```
1. GET /users?email=<email>  →  check if email already exists
   If results are not empty → throw Exception("Email already registered.")

2. POST /users  →  create the new account
   Body: { name, email, password }
   Returns: the newly created UserModel (with server-assigned id)
```

Using a query parameter (?email=value) to filter at the API level is more efficient than fetching all users and filtering in Dart. MockAPI supports basic field filtering through query params.

6.3 fetchBlogs() - [GET]

```
1. GET /blogs  →  returns array of all blog objects
2. Each item is parsed into a BlogModel via fromJson()
3. The list is sorted by createdAt descending (newest first):
   blogs.sort((a, b) => b.createdAt.compareTo(a.createdAt))
```

Sorting is done in Dart rather than relying on API ordering, as MockAPI does not guarantee sort order on GET all.

6.4 addBlog() - [POST]

```
POST /blogs
Body: {
  'blog_name': title,
  'content': content,
  'Author_name': author,
  'createdAt': DateTime.now().toIso8601String()
}
Expects: 200 or 201 → returns the created BlogModel with server-assigned id
```

6.5 updateBlog() - [PUT]

```
PUT /blogs/{id}
Body: { blog_name, content, Author_name, createdAt }
The original createdAt is preserved (passed from the old blog object)
Expects: 200 → returns the updated BlogModel
```

6.6 deleteBlog() - [DELETE]

```
DELETE /blogs/{id}
Expects: 200 or 204
Returns: void - controller removes the item from the local list
```

7. State Management - GetX Controllers

GetX uses a reactive pattern. Observable variables are declared with the .obs suffix. Any widget wrapped in Obx(()=> ...) automatically re-renders when those variables change. This removes the need for setState() or manually calling notifyListeners().

7.1 AuthController

Located at lib/controllers/auth_controller.dart. Manages authentication state for the entire app.

Property / Method	What It Does
isLoading (RxBool)	Tracks whether an API call is in progress. Drives the loading spinner on the Login/Register buttons via Obx.
currentUser (Rxn<UserModel>)	Holds the currently logged-in user, or null if not logged in. Rxn is a nullable Rx variable.
isLoggedIn (bool getter)	Returns true if currentUser.value is not null. Can be used for route guards.
userName (String getter)	Returns the user's name for the greeting on HomeScreen, or "User" as fallback.
login()	Calls ApiService.login(), sets currentUser on success, navigates to home, or shows an error snackbar.
register()	Calls ApiService.register(), on success navigates back to login with a success message.
logout()	Sets currentUser to null, navigates to login screen, shows logout confirmation.

Error handling pattern used in AuthController

Every async method uses a try / catch / finally block. The isLoading flag is set to true at the start and always reset to false in the finally block - even when an exception is thrown. This prevents the button from getting stuck in a loading state if the network call fails.

7.2 BlogController

Located at lib/controllers/blog_controller.dart. Manages the entire blog list and all CRUD operations.

Property / Method	What It Does
isLoading (RxBool)	Shows/hides loading indicators across blog screens.
blogs (RxList<BlogModel>)	The live list of blogs. Any Obx watching this list re-renders when blogs are added, removed, or updated.
onInit()	Called automatically by GetX when the controller is first created. Triggers fetchBlogs() so the app loads data immediately on startup.
fetchBlogs()	Calls ApiService.fetchBlogs() and replaces the local list using blogs.assignAll(). assignAll() triggers a reactive update in one operation.
addBlog()	Calls ApiService.addBlog() and inserts the new blog at index 0 (top of list) using blogs.insert(0, newBlog), then calls Get.back() to return to the blogs list.
updateBlog()	Calls ApiService.updateBlog(), then finds the old blog in the list by id using indexWhere() and replaces it in-place so the UI updates without a full reload.
deleteBlog()	Calls ApiService.deleteBlog(), then removes the item using blogs.removeWhere() which updates the UI reactively.

8. Screens

8.1 LoginScreen

Located at lib/screens/login_screen.dart. The initial screen of the app.
validateEmail(): Checks that the field is not empty and uses GetX's built-in GetUtils.isEmail() to verify the email format.

validatePassword(): Checks that the field is not empty and has at least 6 characters.

submitLogin(): Calls formKey.currentState!.validate() first. If all validators pass (return null), it calls authController.login().

The AuthHeader widget renders the wave-clipped gradient banner at the top of the page.

The password field has a show/hide toggle managed by a local boolean hidePassword via setState().

A "Remember Me" checkbox is shown as a UI element - persistence can be added later via shared_preferences.

The Google login button is present for UI completeness but shows a snackbar noting it is not implemented in this version.

8.2 RegisterScreen

Located at lib/screens/register_screen.dart. Collects name, email, password, and confirm password.

validateConfirmPassword(): Cross-validates against the password field's current value to ensure both match. This is why confirmPasswordController validates against passwordController.text directly.

On successful registration, the user is sent back to the login screen (not auto-logged in), which is a standard UX practice.

All four TextEditingController instances are disposed in dispose() to prevent memory leaks.

8.3 HomeScreen

Located at lib/screens/home_screen.dart. The main landing screen after login. Shows a personalised greeting and a scrollable feed of all blogs.

lib/screens/home_screen.dart. The main landing screen after login. Shows a personalised greeting and a scrollable feed of all blogs.
Reactive loading: The entire body is inside Obx(), so it re-renders whenever blogController.isLoading or blogController.blogs changes.
itemCount trick: When blogs exist, itemCount is blogs.length + 1 because index 0 is reserved for the header widget (_HomeHeader). When the list is empty, itemCount is 2 (header + empty state widget). This avoids duplicating the header outside the ListView.
Pull-to-refresh: Wrapped in a RefreshIndicator that calls blogController.fetchBlogs().Tapping a blog card navigates to BlogDetailScreen, passing the BlogModel via Get.arguments.

8.4 BlogsScreen

Located at lib/screens/blogs_screen.dart. The management screen where the user can edit and delete their blogs.

lib/screens/blogs_screen.dart. The management screen where the user can edit and delete their blogs.

showDeleteDialog(): Shows an AlertDialog with Cancel and Delete buttons using Get.dialog(). Delete is only executed after the user confirms - Cancel simply calls Get.back() without deleting anything.

Blogs are loaded once via BlogController.onInit(). A refresh icon in the header lets the user force a reload manually.
Each blog is rendered by the BlogCard widget, which receives onEdit and onDelete as callbacks.
Floating Action Button: Navigates to AddBlogScreen on press. The FAB uses a gradient Container with a transparent FloatingActionButton inside it to achieve a gradient effect, since Flutter's FAB does not natively support gradients.

8.5 AddBlogScreen

Located at lib/screens/add_blog_screen.dart. A form with two fields: Blog Title and Blog Content.

lib/screens/add_blog_screen.dart. A form with two fields: Blog Title and Blog Content.

validateTitle(): Requires non-empty input and a minimum of 3 characters.
validateContent(): Requires non-empty input and a minimum of 10 characters.
submitBlog(): Validates the form, then calls blogController.addBlog(title, content). The author name is pulled automatically from authController.currentUser.value?.name inside the controller, so no separate author field is needed on the form.The Obx wrapper around GradientButton reacts to blogController.isLoading to display a spinner while the blog is being submitted.

8.6 EditBlogScreen

Located at lib/screens/edit_blog_screen.dart. Pre-fills the title and content with the selected blog's existing data.

lib/screens/edit_blog_screen.dart. Pre-fills the title and content with the selected blog's existing data.

initState(): The existing blog is received from the previous screen via Get.arguments. The text controllers are initialised as TextEditingController(text: blog.title) and TextEditingController(text: blog.content) so both fields are pre-filled when the screen opens.
submitUpdate(): Passes the oldBlog object (which carries the original id, author, and createdAt) along with the updated title and content to blogController.updateBlog(). The controller preserves the original author name and creation timestamp.

8.7 BlogDetailScreen

Located at lib/screens/blog_detail_screen.dart. A read-only view of a single blog post. Receives the BlogModel via Get.arguments from the HomeScreen.

lib/screens/blog_detail_screen.dart. A read-only view of a single blog post. Receives the BlogModel via Get.arguments from the HomeScreen.Formats the date using DateFormat("MMM dd, yyyy hh:mm a") from the intl package, for example: "May 14, 2025 03:30 PM".Displays a CircleAvatar with the first letter of the author's name as a compact visual identifier.The full blog content is displayed without truncation, with a lineHeight of 1.6 for comfortable readability.

9. Reusable Widgets

9.1 AppTextField

Located at lib/widgets/app_text_field.dart. A custom TextFormField wrapper used across all forms in the app.

9.2 GradientButton

Located at lib/widgets/gradient_button.dart. A gradient-styled primary action button used for Login, Register, Post Blog, and Update Blog.

9.3 AuthHeader

Located at lib/widgets/auth_header.dart. The decorative wave-clipped gradient header shown on the Login and Register screens.

9.4 BlogCard

Located at lib/widgets/blog_card.dart. The list item card shown in BlogsScreen. Accepts the blog object plus onEdit and onDelete callbacks.

9.5 AppBottomNavBar

Located at lib/widgets/app_bottom_nav_bar.dart. A custom bottom navigation bar shown on HomeScreen and BlogsScreen.

10. Utilities

10.1 AppConstants, AppColors, AppTextStyles, AppGradients, AppRadius

All defined in lib/utils/app_constants.dart. Centralising design tokens means changing a colour or font size in one place updates it everywhere in the app.

Class	Contains
AppConstants	App name string ("Blog App") and the MockAPI base URL.
AppColors	All colour values used across the app - primary, background, text, error, success, shadow, border.
AppGradients	The primary LinearGradient (secondary → primary, top-left to bottom-right) used on buttons, FABs, and the auth header.
AppRadius	Border radius constants: small (10), medium (14), large (18), extraLarge (24).
AppTextStyles	Predefined TextStyle objects: headingLarge, headingMedium, title, body, small, button.
AppDurations	Animation duration constants: fast (200ms), normal (350ms).

10.2 ResponsiveContext Extension

Also in app_constants.dart. A Dart extension on BuildContext that provides responsive sizing methods.

This extension is used throughout all screens and widgets like context.responsiveWidth(24) instead of hardcoded pixel values, making the UI adapt naturally to different screen sizes.

10.3 AppRoutes

Located at lib/utils/app_routes.dart. Defines all named route strings as static constants.

```
static const String login     = '/login';
static const String register  = '/register';
static const String home      = '/home';
static const String blogs     = '/blogs';
```

```
static const String addBlog    = '/add-blog';
static const String editBlog   = '/edit-blog';
static const String blogDetail = '/blog-detail';
```

10.4 ApiEndpoints

Located at lib/utils/api_endpoints.dart. Keeps the URL path segments separate from the base URL.

```
static const String users = '/users';
static const String blogs = '/blogs';
```

11. App Flow

The following describes the complete user journey through the app:

11.1 Authentication Flow

- App launches → Login screen is shown (initial route: /login).
- User enters email and password → taps Login.
- Form validates both fields. If validation fails, error text appears beneath the field.
- If valid, AuthController.login() is called → ApiService fetches all users → searches for a match.
- On match → currentUser is set → app navigates to /home using Get.offAllNamed() (clears back stack).
- On no match → GetX snackbar shows "Invalid email or password."

11.2 Registration Flow

- From Login screen, tap "Register" → navigates to /register.
- User fills in name, email, password, and confirm password → taps Register.
- All four fields are validated. Confirm password must match password field.
- ApiService checks if email already exists via query param GET.
- If email is free → POST /users → new user created → redirect to /login with success message.
- If email already exists → error snackbar is shown.

11.3 Blog Management Flow

- After login, HomeScreen loads and BlogController.fetchBlogs() runs automatically (via onInit).
- Blogs are fetched from /blogs, sorted newest-first, and displayed as cards.
- Tapping a blog card opens BlogDetailScreen (full content view).
- Navigating to the Blogs tab (bottom nav) shows BlogsScreen with edit/delete options per card.
- Tapping the FAB (+) opens AddBlogScreen → fill title and content → tap "Post Blog".
- New blog is POSTed to /blogs → inserted at top of local list → screen navigates back automatically.
- Tapping Edit on a card → EditBlogScreen opens pre-filled → modify → tap "Update Blog" → PUTs to API → list updates in place.
- Tapping Delete → confirmation dialog appears → confirm → DELETEs from API → removed from local list.

12. Error Handling & Validation

12.1 Form Validation

Every form in the app uses Flutter's built-in Form + GlobalKey<FormState> pattern. Validators are individual functions defined per field, keeping them easy to read and test individually. Here is a summary of all validation rules applied:

Screen	Field	Validation Rules
Login	Email	Required · Must pass GetUtils.isEmail()
Login	Password	Required · Minimum 6 characters
Register	Name	Required · Minimum 3 characters
Register	Email	Required · Must pass GetUtils.isEmail()
Register	Password	Required · Minimum 6 characters
Register	Confirm Password	Required · Must match password field exactly
Add Blog	Title	Required · Minimum 3 characters
Add Blog	Content	Required · Minimum 10 characters
Edit Blog	Title	Required · Minimum 3 characters
Edit Blog	Content	Required · Minimum 10 characters

12.2 API Error Handling

All methods in ApiService throw typed exceptions for non-success HTTP status codes. All controller methods catch these exceptions and display user-friendly messages via Get.snackbar() instead of letting the error propagate to the UI layer.

12.3 Delete Confirmation

Rather than immediately deleting when the user taps the delete button, the app shows a confirmation dialog. This is a UX safety net for a destructive action. The dialog is created with Get.dialog(AlertDialog(...)) and only proceeds to call blogController.deleteBlog(id) after the user confirms.

13. Design System

The app uses a consistent design language throughout, defined in app_constants.dart and applied via the reusable widgets described in Section 9.

Design Token	Value
Primary colour	#5B5FEF (indigo)
Secondary colour	#7B61FF (purple)
Background	#F8F8FD (off-white)

Card background	#FFFFFF (white)
Dark text	#111827
Light/grey text	#6B7280
Error	#EF4444 (red)
Success	#22C55E (green)
Border radius - small	10px
Border radius - medium	14px
Border radius - large	18px
Card shadow	black at 6% opacity, blur 16
Button gradient	linear: #7B61FF → #5B5FEF

14. Summary

The Blog Management App is a well-structured Flutter application that demonstrates how GetX can be used for state management, dependency injection, and routing without adding excessive complexity to the codebase. Key strengths of the implementation include:

- Clean separation of concerns - services handle API calls, controllers handle logic, screens handle UI.
- Fully reactive UI - any change to an Rx variable in a controller immediately updates all Obx widgets listening to it.
- Responsive layout - the ResponsiveContext extension ensures the UI scales appropriately across small phones, normal phones, and tablets.
- Defensive data parsing - BlogModel.fromJson handles multiple possible field name variations and uses safe fallbacks for all fields.
- User experience details - keyboard dismissal on tap outside, pull-to-refresh, delete confirmation dialogs, loading state on buttons, and automatic navigation after form submission.
- Centralised design tokens - all colours, text styles, radii, and gradients are in one file, making design changes straightforward.

The only areas intentionally omitted are Google Sign-In, Forgot Password, and the Profile screen -these are shown as UI placeholders with snackbars indicating they are not included in this version.